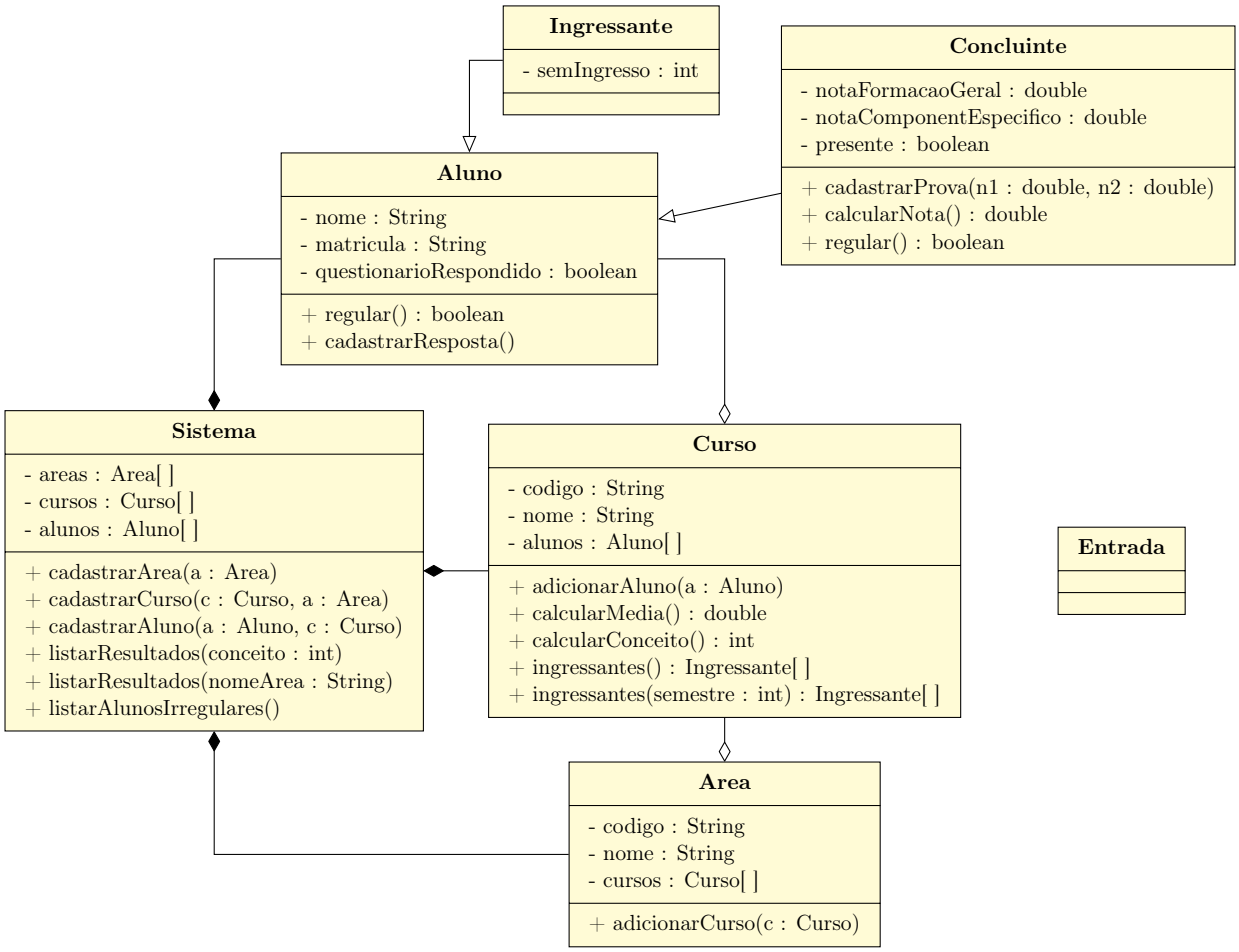
Atividade 2: Implementação

POO - BSI - Ifes Serra

14 de outubro de 2026

1 Diagrama de Classes UML

Nesta atividade, você deve começar a implementar o sistema feito na Atividade 1. Como acontece em vários projetos, a etapa de implementação demandou que alguns ajustes fossem feitos no diagrama de classes. Por exemplo, optou-se por não armazenar mais as perguntas do questionários (já que a Instituição só sabe se o aluno respondeu ao questionário, e não quais suas respostas). O diagrama ainda pode vir a mudar na próxima etapa (a fase final), em que serão adicionados os conceitos de tratamento de erros e exceções, persistência em arquivos, classes abstratas, interfaces, coleções e ordenação. Por enquanto, as classes e relacionamentos a serem implementados estão mostrados no diagrama a seguir:



Note que há uma nova classe chamada Entrada, que vai cuidar de **toda a interação** com o usuário do sistema. **Nenhuma leitura de dados ou impressão pode ser feita fora da classe Entrada**. A seguir, uma descrição detalhada dos atributos e métodos que devem ser implementados em cada classe por enquanto.

2 Atributos e Métodos

As classes **Aluno**, **Ingressante**, **Concluinte**, **Curso** e **Area** possuem um método **toString**. O método **toString** em uma classe Java qualquer indica como um objeto desta classe será representado quando for impresso. Na classe **Aluno**, o método **toString** deverá exibir nome e matrícula do aluno, no formato a seguir:

```
Pedro da Silva (20262BSI001)
```

O método nas classes **Ingressante** e **Concluinte** reescrevem o método da superclasse, informando se o aluno é um ingressante ou concluinte. Exemplos:

```
Pedro da Silva (20262BSI001) - INGRESSANTE
Valentina de Souza (20202BSI021) - CONCLUINTE
```

Na classe **Curso**, o método **toString** deverá mostrar o nome e código do curso:

```
Sistemas de Informação - COMP001
```

Já na classe **Area**, o método **toString** deverá mostrar o nome e código da área:

```
Informática - 001
```

Os construtores e os métodos getters e setters serão omitidos de todos os diagramas. Crie os que forem necessários.

Um **Aluno** possui **nome**, **matrícula** e um booleano indicando se ele já respondeu ao questionário. Os alunos são criados antes que eles possam ter preenchido o questionário. O método **regular** informa se ele está regular no ENADE. A princípio, ele está regular se respondeu ao questionário (mas isso pode ser redefinido de forma mais específica para o aluno concluinte).

Além das informações de um aluno, o **Ingressante** armazena também o semestre em que ele entrou no curso.

Além das informações de um aluno, o **Concluinte** armazena também sua nota nas questões de formação geral, suas notas nas questões de conhecimento específico e um booleano indicando se ele realizou a prova. Quando o concluinte é criado no sistema, ainda não se sabe suas notas e se ele realizou a prova. Há um método para inserir estas notas posteriormente, indicando também que ele esteve presente. Há ainda um método para calcular sua nota na prova (média ponderada: peso 3 para a nota das questões de formação geral e peso 7 para a nota das questões de componentes específicos). Além disso, para um concluinte estar regular, além dos requisitos necessários do **Aluno**, ele precisa também ter participado da prova.

Um **Curso** armazena seu código, nome e conjunto de alunos. A média do curso é a média aritmética simples da nota de todos os alunos concluintes do curso. Já o conceito ENADE transforma a média em um inteiro entre 1 e 5:

- Conceito **1**: média < 2.0
- Conceito **2**: $2.0 \leq$ média < 4.0
- Conceito **3**: $4.0 \leq$ média < 6.0
- Conceito **4**: $6.0 \leq$ média < 8.0
- Conceito **5**: média ≥ 8.0

O médoto **ingressantes** de um curso retorna um conjunto com todos os ingressantes deste curso. Caso seja passado como parâmetro um inteiro contendo um semestre em específico, o método retorna apenas os ingressantes que entraram naquele semestre. Como o conjunto de alunos armazena tanto ingressantes como concluintes, pode ser necessário saber a classe de cada elemento. Em Java, para descobrir se um objeto **x** pertence a uma classe **C**, podemos fazer:

```
if (x instanceof C) {}
```

Suponha que uma classe **Gerente** herda da classe **Funcionario**, e a subclasse possui um método chamado **validarAcesso** que não existe na superclasse. Se quisermos validar o acesso de todos os gerentes, mas o sistema armazena um único conjunto chamado **funcionarios** contendo todos os objetos que são tanto gerentes quanto funcionários, podemos fazer:

```
for (Funcionario f : funcionarios) {
    if (f instanceof Gerente) {
        Gerente g = (Gerente) f; // Conversao explicita
        g.validarAcesso();
    }
}
```

Outra opção é:

```
for (Funcionario f : funcionarios) {
    if (f instanceof Gerente g) {
        g.validarAcesso();
    }
}
```

Por fim, se o método **listarResultados** da classe **Sistema** receber um conceito, ele exibe todos os cursos que obtiveram aquele conceito no ENADE. Já se ele receber o nome de uma área, exibe o conceito de todos os cursos daquela área.

3 A Classe Entrada

Por fim, a classe Entrada possui métodos estáticos para cuidar de toda a interação com o usuário. Alguns métodos já foram implementados no arquivo *Entrada.java* disponibilizado. Os métodos descritos para esta classe são **sugestões**. O importante é que todas as funcionalidades do menu sejam implementadas nesta classe.

Quando o sistema é iniciado, é exibido o *menu* do sistema:

```
Escolha uma opção:
1) Cadastro
2) Relatórios
0) Sair
```

As opções de cadastro são:

```
Escolha uma opção:
1) Cadastrar Área
2) Cadastrar Curso
3) Cadastrar Aluno Ingressante
4) Cadastrar Aluno Concluinte
5) Cadastrar Resposta de Questionário
6) Cadastrar Resultado de Prova
0) Voltar
```

A opção **Cadastrar Área** lê os dados de uma nova área, **nesta ordem**:

```
Digite o nome da área: Informática
Digite o código da área: 001
Área cadastrada com sucesso!
```

A opção **Cadastrar Curso** lê os dados de um novo curso, **nesta ordem**:

```
Digite o nome do curso: Sistemas de Informação
Digite o código do curso: INF001
Áreas cadastradas:
Informática (001)
Comunicação (002)
Digite o código da área do curso: 001
Curso cadastrado com sucesso!
```

A opção **Cadastrar Aluno Ingressante** lê os dados de um ingressante, **nesta ordem**:

```
Digite o nome do ingressante: Enzo da Silva
Digite a matrícula do ingressante: 20261BSI001
Digite o semestre de ingresso: 1
Cursos cadastrados:
Sistemas de Informação (INF001)
Ciência da Computação (INF002)
Digite o código do curso do aluno: INF001
Aluno ingressante cadastrado com sucesso!
```

A opção **Cadastrar Aluno Concluinte** lê os dados de um ingressante, **nesta ordem**:

```
Digite o nome do concluinte: Valentina de Souza
Digite a matrícula do concluinte: 20202BSI021
Cursos cadastrados:
Sistemas de Informação (INF001)
Ciência da Computação (INF002)
Digite o código do curso do aluno: INF001
Aluno concluinte cadastrado com sucesso!
```

A opção **Cadastrar Resposta de Questionário** exibe os alunos que ainda não responderam ao questionário, e permite indicar que ele o respondeu, **nesta ordem**:

```
Alunos cadastrados:
Enzo da Silva (20261BSI001) - INGRESSANTE
Pedro da Silva (20262BSI002) - INGRESSANTE
Valentina de Souza (20202BSI021) - CONCLUINTE
Ana de Souza (20201BSI030) - CONCLUINTE
Digite a matrícula do aluno que respondeu ao questionário: 20261BSI001
Resposta cadastrada com sucesso!
```

Por fim, a opção **Cadastrar Resultado de Prova** exibe os concluintes que ainda **não** participaram da prova, e permite indicar as notas da avaliação de um deles, **nesta ordem**:

```
Concluintes cadastrados:
Valentina de Souza (20202BSI021) - CONCLUINTE
Ana de Souza (20201BSI030) - CONCLUINTE
Digite a matrícula do aluno que realizou a prova: 20202BSI021
Digite o valor da nota de formação geral: 8.5
Digite o valor da nota de conhecimentos específicos: 7.0
Prova cadastrada com sucesso!
```

Ao alterar as notas, lembre-se de indicar também que ele realizou a prova.

As opções do menu de relatórios são:

```
1) Listar Resultados por Conceito
2) Listar Resultados por Área
3) Listar Alunos Irregulares
0) Voltar
```

A opção **Listar Resultados por Conceito** pede que o usuário escolha um conceito (de 1 a 5) e exibe todos os cursos com aquele conceito:

```
Digite o conceito desejado (1 a 5): 5
Cursos com conceito 5:
Sistemas de Informação (INF001) - Área: Informática (001)

Digite o conceito desejado (1 a 5): 4
Cursos com conceito 4:
Nenhum curso com conceito 4 foi encontrado.
```

A opção **Listar Resultados por Área** pede que o usuário digite o **nome** de uma área, e exibe os conceitos de todos os cursos daquela área:

```
Áreas cadastradas:
Informática (001)
Comunicação (002)
Digite o nome da área: Informática
Conceitos dos cursos da área Informática (001):
Sistemas de Informação (INF001): 5
Ciência da Computação (INF002): 1

Áreas cadastradas:
Informática (001)
Comunicação (002)
Digite o nome da área: 001
Nenhum curso na área 001 foi encontrado.
```

Por fim, a opção **Listar Alunos Irregulares** exibe todos os alunos irregulares (aqueles que não responderam o questionário e, no caso de concluintes, que não realizaram a prova):

```
Alunos irregulares com o ENADE:
Pedro da Silva (20262BSI002) - INGRESSANTE - CURSO: Sistemas de Informação (INF001)
Ana de Souza (20201BSI030) - CONCLUINTE - CURSO: Sistemas de Informação (INF001)
```

Se necessário, crie métodos **getters** para acessar o valor de algum atributo que esteja encapsulado nas classes. Para cada conjunto de objetos no diagrama, o aluno deve decidir se é mais vantajoso implementá-lo usando **array** ou **ArrayList**. Apenas estas duas estruturas são permitidas nesta atividade.

É importante ler os dados de cadastros e opções do menu **na ordem em que foram descritos neste documento**, pois a correção será automatizada. Na página da disciplina, há um arquivo "input.txt" com um exemplo de entrada do arquivo. Basta rodar seu programa com esse arquivo na pasta onde o programa está sendo executado, e o arquivo será lido automaticamente (ao invés do programa ler a entrada do teclado). Faça isso apenas quando todas as opções do seu menu já estiverem funcionando.

4 Observações

1. A atividade vale 40 pontos e deve ser entregue até 27/10.
2. O código deve ser feito em Java, considerando as boas práticas de POO (como reuso de código, por exemplo, sempre que possível).
3. A atividade pode ser feita em grupos de até dois integrantes.
4. Atividades entregues após o prazo serão automaticamente rejeitados.
5. Atividades com erro de execução, com formato de saída incorreto, ou que não compilarem terão nota 0.
6. Atividades considerados plágio terão nota 0 para quem copiou e para quem forneceu o exercício, e serão enviados ao Conselho de Ética.
7. A Atividade deve ser enviado na sala da disciplina do AVA.
8. Em caso de dúvidas na especificação da atividade ou na própria atividade, contate-me em sala de aula.